Be Competitive Or Compro

Geek Week 2021 is final

GeeksforGeeks

Square On
for restaura

Learn mor







Your Career Is Not A

College Students Should Brush

Squa

GeeksforGeeks

400+ TOP COMPILER DESIGN LAB VIVA Questions and Answers

MPILER DESIGN VIVA Questions :-

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

A compiler is a program that reads a program written in one language –the source language and translates it into an equivalent program in another language-the target language. The compiler reports to its user the presence of errors in the source program.

2. What are the two parts of a compilation? Explain briefly.

- Analysis and Synthesis are the two parts of compilation.
- The analysis part breaks up the source program into constituent pieces and creates an intermediate representation of the source program.
- The synthesis part constructs the desired target program from the intermediate representation.

3. List the subparts or phases of analysis part.

Analysis consists of three phases:

- Linear Analysis.
- Hierarchical Analysis.
- Semantic Analysis.

4. Depict diagrammatically how a language is processed.

Skeletal source program

↓

Preprocessor

↓

Source program

↓

Compiler



Target assembly program

↓

Assembler

↓

Relocatable machine code

↓

Loader/ link editor ←library, relocatable object files

↓

Absolute machine code

5. What is linear analysis?

Linear analysis is one in which the stream of characters making up the source program is read from left to right and grouped into tokens that are sequences of characters having a collective meaning.

Also called lexical analysis or scanning.

6. List the various phases of a compiler.

The following are the various phases of a compiler:

1. Lexical Analyzer
2. Syntax Analyzer
3. Semantic Analyzer
4. Intermediate code generator
5. Code optimizer
6. Code generator

7. What are the classifications of a compiler?



1. Single- pass
2. Multi-pass
3. Load-and-go
4. Debugging or optimizing

8. What is a symbol table?

A symbol table is a data structure containing a record for each identifier, with fields for the attributes of the identifier. The data structure allows us to find the record for each identifier quickly and to store or retrieve data from that record quickly.

Whenever an identifier is detected by a lexical analyzer, it is entered into the symbol table. The attributes of an identifier cannot be determined by the lexical analyzer.

9. Mention some of the cousins of a compiler.

Cousins of the compiler are:

- Preprocessors
- Assemblers
- Loaders and Link-Editors

10. List the phases that constitute the front end of a compiler.

The front end consists of those phases or parts of phases that depend primarily on the source language and are largely independent of the target machine. These include

- Lexical and Syntactic analysis
- The creation of symbol table
- Semantic analysis
- Generation of intermediate code
  A certain amount of code optimization can be done by the front end as well. Also includes error handling that goes along with each of these phases.

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

The back end of compiler includes those portions that depend on the target machine and generally those portions do not depend on the source language, just the intermediate language. These include

- Code optimization
- Code generation, along with error handling and symbol- table operations.

12. Define compiler-compiler.

Systems to help with the compiler-writing process are often been referred to as compiler-compilers, compiler-generators or translator-writing systems.

Largely they are oriented around a particular model of languages , and they are suitable for generating compilers of languages similar model.

13. List the various compiler construction tools.

The following is a list of some compiler construction tools:

1. Parser generators
2. Scanner generators
3. Syntax-directed translation engines
4. Automatic code generators
5. Data-flow engines

14. Differentiate tokens, patterns, lexeme.

1. Tokens- Sequence of characters that have a collective meaning.
2. Patterns- There is a set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a pattern associated with the token
3. Lexeme- A sequence of characters in the source program that is matched by the pattern for a token.

15. List the operations on languages.



- Kleene Closure – L* (zero or more concatenations of L)
- Positive Closure – L+ (one or more concatenations of L)

16. Write a regular expression for an identifier.

An identifier is defined as a letter followed by zero or more letters or digits.

The regular expression for an identifier is given as

letter (letter | digit)*

17. Mention the various notational shorthands for representing regular expressions.

- One or more instances (+)
- Zero or one instance (?)
- Character classes ([abc] where a,b,c are alphabet symbols denotes the regular expressions a | b | c.)
- Non regular sets

18. What is the function of a hierarchical analysis?

Hierarchical analysis is one in which the tokens are grouped hierarchically into nested collections with collective meaning.

Also termed as Parsing.

19. What does a semantic analysis do?

Semantic analysis is one in which certain checks are performed to ensure that components of a program fit together meaningfully.

Mainly performs type checking.

20. List the various error recovery strategies for a lexical analysis.



- Deleting an extraneous character
- Inserting a missing character
- Replacing an incorrect character by a correct character
- Transposing two adjacent characters

Compiler Design – Set 2

21. Define parser.
Hierarchical analysis is one in which the tokens are grouped hierarchically into nested collections with collective meaning.

Also termed as Parsing.

22. Mention the basic issues in parsing.
There are two important issues in parsing.

1. Specification of syntax
2. Representation of input after parsing.

23. Why lexical and syntax analyzers are separated out?
Reasons for separating the analysis phase into lexical and syntax analyzers:

Simpler design.
Compiler efficiency is improved.
Compiler portability is enhanced.



- V is a set of non terminals
- T is a set of terminals
- S is a start symbol
- P is a set of production rules

G can be represented as G = (V,T,S,P)

Production rules are given in the following form

Non terminal → (V U T)*

25. Explain the concept of derivation.
Derivation from S means generation of string w from S. For constructing derivation two things are important.

i) Choice of non terminal from several others.

ii) Choice of rule from production rules for corresponding non terminal.

Instead of choosing the arbitrary non terminal one can choose

i) either leftmost derivation – leftmost non terminal in a sentinel form

ii) or rightmost derivation – rightmost non terminal in a sentinel form

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

26. Define ambiguous grammar.
A grammar G is said to be ambiguous if it generates more than one parse tree for some sentence of language L(G).

i.e. both leftmost and rightmost derivations are same for the given sentence.

27. What is a operator precedence parser?
A grammar is said to be operator precedence if it possess the following properties:

1. No production on the right side is ε.

2. There should not be any production rule possessing two adjacent non terminals at the right hand side.

28. List the properties of LR parser.
1. LR parsers can be constructed to recognize most of the programming languages for which the context free grammar can be written.

2. The class of grammar that can be parsed by LR parser is a superset of class of grammars that can be parsed using predictive parsers.

3. LR parsers work using non backtracking shift reduce technique yet it is efficient one.

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

1. SLR parser- simple LR parser
2. LALR parser- lookahead LR parser
3. Canonical LR parser

30. What are the problems with top down parsing?
The following are the problems associated with top down parsing:

- Backtracking
- Left recursion
- Left factoring
- Ambiguity

31. Write the algorithm for FIRST and FOLLOW.
FIRST

1. If X is terminal, then FIRST(X) IS {X}.

2. If X → ε is a production, then add ε to FIRST(X).

3. If X is non terminal and X → Y1,Y2..Yk is a production, then place a in FIRST(X) if for some i , a is in FIRST(Yi) , and ε is in all of FIRST(Y1),...FIRST(Yi-1);

FOLLOW

1. Place $ in FOLLOW(S),where S is the start symbol and $ is the input right endmarker.

2. If there is a production A → αBβ, then everything in FIRST(β) except for ε is placed in FOLLOW(B).

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

3. If there is a production A → αB, or a production A→ αBβ where FIRST(β) contains ε , then everything in FOLLOW(A) is in FOLLOW(B).

32. List the advantages and disadvantages of operator precedence parsing.

Advantages

1. This typeof parsing is simple to implement.

Disadvantages

1. The operator like minus has two different precedence(unary and binary).Hence it is hard to handle tokens like minus sign.

2. This kind of parsing is applicable to only small class of grammars.

33. What is dangling else problem?
Ambiguity can be eliminated by means of dangling-else grammar which is show below:

stmt → if expr then stmt

| if expr then stmt else stmt

| other

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

YACC stands for Yet Another Compiler Compiler which is basically the utility available from UNIX.

Basically YACC is LALR parser generator.

It can report conflict or ambiguities in the form of error messages.

35. What is meant by handle pruning?

A rightmost derivation in reverse can be obtained by handle pruning.

If w is a sentence of the grammar at hand, then w = γn, where γn is the nth right-sentential form of some as yet unknown rightmost derivation

S = γ0 => γ1...=> γn-1 => γn = w

36. Define LR(0) items.

An LR(0) item of a grammar G is a production of G with a dot at some position of the right side. Thus, production A → XYZ yields the four items

A→.XYZ

A→X.YZ

A→XY.Z

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

37. What is meant by viable prefixes?

The set of prefixes of right sentential forms that can appear on the stack of a shift-reduce parser are called viable prefixes. An equivalent definition of a viable prefix is that it is a prefix of a right sentential form that does not continue past the right end of the rightmost handle of that sentential form.

38. Define handle.

A handle of a string is a substring that matches the right side of a production, and whose reduction to the nonterminal on the left side of the production represents one step along the reverse of a rightmost derivation.

A handle of a right – sentential form γ is a production A→β and a position of γ where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ. That is , if S =>αAw =>αβw,then A→β in the position following α is a handle of αβw.

39. What are kernel & non-kernel items?

Kernel items, whish include the initial item, S'→ .S, and all items whose dots are not at the left end.

Non-kernel items, which have their dots at the left end.

40. What is phrase level error recovery?

Phrase level error recovery is implemented by filling in the blank entries in the predictive parsing table with pointers to error routines. These routines may change, insert, or delete symbols on the input and issue appropriate error messages. They may also pop from the stack.

Compiler Design – Set 3

41. What are the benefits of intermediate code generation?

- A Compiler for different machines can be created by attaching different back end to the existing front ends of each machine.

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

- A machine independent code optimizer can be applied to intermediate code in order to optimize the code generation.

42. What are the various types of intermediate code representation?

There are mainly three types of intermediate code representations.

- Syntax tree
- Postfix
- Three address code

43. Define backpatching.

Backpatching is the activity of filling up unspecified information of labels using appropriate semantic actions in during the code generation process.In the semantic actions the functions used are mklist(i),merge_list(p1,p2) and backpatch(p,i)

44. Mention the functions that are used in backpatching.

- mklist(i) creates the new list. The index i is passed as an argument to this function where I is an index to the array of quadruple.
- merge_list(p1,p2) this function concatenates two lists pointed by p1 and p2. It returns the pointer to the concatenated list.
- backpatch(p,i) inserts i as target label for the statement pointed by pointer p.

45. What is the intermediate code representation for the expression a or b and not c?

The intermediate code representation for the expression a or b and not c is the three address sequence

t1 := not c

t2 := b and t1

t3 := a or t2

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

The three address statements can be implemented using the following methods.

- **Quadruple :** a structure with atmost four fields such as operator(OP),arg1,arg2,result.
- **Triples :** the use of temporary variables is avoided by referring the pointers in the symbol table.
- **Indirect triples :** the listing of triples has been done and listing pointers are used instead of using statements.

47. Give the syntax-directed definition for if-else statement.

1. S → if E then S1

E.true := new_label()

E.false :=S.next

S1.next :=S.next

S.code :=E.code | | gen_code(E.true ‘: ‘) | | S1.code

2. S → if E then S1 else S2

E.true := new_label()

E.false := new_label()

S1.next :=S.next

S2.next :=S.next

S.code :=E.code | | gen_code(E.true ‘: ‘) | | S1.code| | gen_code(‘go to’,S.next) | |gen_code(E.false ‘:’) | | S2.code

Compiler Design – Set 4

48. Mention the properties that a code generator should



- The code generator should produce the correct and high quality code. In other words, the code generated should be such that it should make effective use of the resources of the target machine.
- Code generator should run efficiently.
- **Define and use –** the three address statement a:=b+c is said to define a and to use b and c.
- **Live and dead –** the name in the basic block is said to be live at a given point if its value is used after that point in the program. And the name in the basic block is said to be dead at a given point if its value is never used after that point in the program.

49. What is a flow graph?

A flow graph is a directed graph in which the flow control information is added to the basic blocks.

1. The nodes to the flow graph are represented by basic blocks
2. The block whose leader is the first statement is called initial block.
3. There is a directed edge from block B1 to block B2 if B2 immediately follows B1 in the given sequence. We can say that B1 is a predecessor of B2.

50. What is a DAG? Mention its applications.

Directed acyclic graph(DAG) is a useful data structure for implementing transformations on basic blocks.

DAG is used in

- Determining the common sub-expressions.
- Determining which names are used inside the block and computed outside the block.
- Determining which statements of the block could have their computed value outside the block.
- Simplifying the list of quadruples by eliminating the common su-expressions and not performing the assignment of the form x := y unless and until it is a must.



improving target code. This technique is applied to improve the performance of the target program by examining the short sequence of target instructions and replacing these instructions by shorter or faster sequence.

52. List the characteristics of peephole optimization.

1. Redundant instruction elimination
2. Flow of control optimization
3. Algebraic simplification
4. Use of machine idioms

53. How do you calculate the cost of an instruction?
The cost of an instruction can be computed as one plus cost associated with the source and destination addressing modes given by added cost.

MOV R0,R1 1

MOV R1,M 2

SUB 5(R0),*10(R1) 3

54. What is a basic block?
A basic block is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without halt or possibility of branching.

Eg. t1:=a*5

t2:=t1+7

t3:=t2-5

t4:=t1+t3

t5:=t2+b



Compiler Design – Set 5

56. Mention the issues to be considered while applying the techniques for code optimization.

- The semantic equivalence of the source program must not be changed.
- The improvement over the program efficiency must be achieved without changing the algorithm of the program.
- The machine dependent optimization is based on the characteristics of the target machine for the instruction set used and addressing modes used for the instructions to produce the efficient target code.
- The machine independent optimization is based on the characteristics of the programming languages for appropriate programming structure and usage of efficient arithmetic properties in order to reduce the execution time.
- Available expressions
- Reaching definitions
- Live variables
- Busy variables

57. What are the basic goals of code movement?

1. To reduce the size of the code i.e. to obtain the space complexity.
2. To reduce the frequency of execution of code i.e. to obtain the time complexity.

58. List the different storage allocation strategies.

The strategies are:

- Static allocation
- Stack allocation
- Heap allocation

59. What are the contents of activation record?

The activation record is a block of memory used for managing the



- Temporary variables
- Local variables
- Saved machine registers
- Control link
- Access link
- Actual parameters
- Return values

60. What is dynamic scoping?
In dynamic scoping a use of non-local variable refers to the non-local data declared in most recently called and still active procedure. Therefore each time new findings are set up for local names called procedure. In dynamic scoping symbol tables can be required at run time.

61. Define symbol table.
Symbol table is a data structure used by the compiler to keep track of semantics of the variables. It stores information about scope and binding information about names.

62. What is code motion?
Code motion is an optimization technique in which amount of code in a loop is decreased. This transformation is applicable to the expression that yields the same result independent of the number of times the loop is executed. Such an expression is placed before the loop.

63. What are the properties of optimizing compiler?
The source code should be such that it should produce minimum amount of target code.

There should not be any unreachable code.

Dead code should be completely removed from source language.

The optimizing compilers should apply following code improving transformations on source language.

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

iii) code movement

iv) strength reduction

64. What are the various ways to pass a parameter in a function?

1. Call by value
2. Call by reference
3. Copy-restore
4. Call by name

65. Suggest a suitable approach for computing hash function.

Using hash function we should obtain exact locations of name in symbol table.

The hash function should result in uniform distribution of names in symbol table.

The hash function should be such that there will be minimum number of collisions. Collision is such a situation where hash function results in same location for storing the names.

---- >> Related Posts Of Above Questions :::

------>>[MOST IMPORTANT]<<------

1. **300+ TOP COMPILER DESIGN Multiple Choice Questions and Answers**
2. **300+ TOP Compiler Design Interview Questions and Answer**
3. **250+ TOP MCQs on Lexical Analysis and Answers**
4. **250+ TOP MCQs on Syntax Analyser and Answers**
5. **250+ TOP MCQs on YACC Parser Generator and Answers**
6. **250+ TOP MCQs on Cross Compiler and Answers**



8. 250+ TOP MCQs on Top-Down Parsing and Answers
9. 250+ TOP MCQs on Applications – Parsers and Answers
10. 250+ TOP MCQs on Top-Down Parsing and Answers
11. 300+ TOP SYSTEM Programming LAB VIVA Questions and Answers
12. 250+ TOP MCQs on Lexical Analysis and Answers
13. 250+ TOP MCQs on Handle of Right sentinel Grammar and Answers
14. 250+ TOP MCQs on Data Structure for Representing Parsing Table Answers
15. 250+ TOP MCQs on Syntax Analyser and Answers
16. What is cross compiler?
17. 250+ TOP MCQs on Intermediate Code-Generation and Answers
18. 250+ TOP MCQs on The Closure Compiler and Answers
19. 250+ TOP MCQs on CFL- Other Normal Forms and Answers
20. 250+ TOP MCQs on Lexical Analysis and Answers

ONE THOUGHT ON "400+ TOP COMPILER DESIGN LAB VIVA QUESTIONS AND ANSWERS"

san
MARCH 5, 2016 AT 10:26 AM
very easy

REPLY

LEAVE A REPLY

Your email address will not be published. Required fields are marked *

cisco SECURE X Have you heard? Cisco Secure simplifies your security. Integra

Name *

Email *

Website

Post Comment

Engineering 2021 , Engineering Interview Questions.com , Theme by Engineering|| Privacy Policy|| Terms and Conditions|| ABOUT US|| Contact US||
Engineering interview questions,Mcqs,Objective Questions,Class Lecture Notes,Seminor topics,Lab Viva Pdf PPT Doc Book free download. Most Asked Technical Basic CIVIL | Mechanical | CSE | EEE | ECE | IT | Chemical | Medical MBBS Jobs Online Quiz Tests for Freshers Experienced .

